

observes that the string x is input to F , but only the existence of A , k , and the constant factor implicit in the $O(n^k)$ running time is known to F (since the language L belongs to NP), not their actual values. Thus, the professor concludes that F cannot possibly construct the circuit C and that the language CIRCUIT-SAT is not necessarily NP-hard. Explain the flaw in the professor's reasoning.

34.4 NP-completeness proofs

The proof that the circuit-satisfiability problem is NP-complete showed directly that $L \leq_p \text{CIRCUIT-SAT}$ for every language $L \in \text{NP}$. This section shows how to prove that languages are NP-complete without directly reducing *every* language in NP to the given language. We'll explore examples of this methodology by proving that various formula-satisfiability problems are NP-complete. Section 34.5 provides many more examples.

The following lemma provides a foundation for showing that a given language is NP-complete.

Lemma 34.8

If L is a language such that $L' \leq_p L$ for some $L' \in \text{NPC}$, then L is NP-hard. If, in addition, we have $L \in \text{NP}$, then $L \in \text{NPC}$.

Proof Since L' is NP-complete, for all $L'' \in \text{NP}$, we have $L'' \leq_p L'$. By supposition, we have $L' \leq_p L$, and thus by transitivity (Exercise 34.3-2), we have $L'' \leq_p L$, which shows that L is NP-hard. If $L \in \text{NP}$, we also have $L \in \text{NPC}$. ■

In other words, by reducing a known NP-complete language L' to L , we implicitly reduce every language in NP to L . Thus, Lemma 34.8 provides a method for proving that a language L is NP-complete:

1. Prove $L \in \text{NP}$.
2. Prove that L is NP-hard:

- a. Select a known NP-complete language L' .
- b. Describe an algorithm that computes a function f mapping every instance $x \in \{0, 1\}^*$ of L' to an instance $f(x)$ of L .
- c. Prove that the function f satisfies $x \in L'$ if and only if $f(x) \in L$ for all $x \in \{0, 1\}^*$.
- d. Prove that the algorithm computing f runs in polynomial time.

This methodology of reducing from a single known NP-complete language is far simpler than the more complicated process of showing directly how to reduce from every language in NP. Proving CIRCUIT-SAT $\in$ NPC furnishes a starting point. Knowing that the circuit-satisfiability problem is NP-complete makes it much easier to prove that other problems are NP-complete. Moreover, as the catalog of known NP-complete problems grows, so will the choices for languages from which to reduce.

Formula satisfiability

To illustrate the reduction methodology, let's see an NP-completeness proof for the problem of determining whether a boolean *formula*, not a *circuit*, is satisfiable. This problem has the historical honor of being the first problem ever shown to be NP-complete.

We formulate the *(formula) satisfiability* problem in terms of the language SAT as follows. An instance of SAT is a boolean formula ϕ composed of

1. n boolean variables: $x_1, x_2, \dots, x_n$;
2. m boolean connectives: any boolean function with one or two inputs and one output, such as $\wedge$ (AND), $\vee$ (OR), $\neg$ (NOT), $\rightarrow$ (implication), $\leftrightarrow$ (if and only if); and
3. parentheses. (Without loss of generality, assume that there are no redundant parentheses, i.e., a formula contains at most one pair of parentheses per boolean connective.)

We can encode a boolean formula ϕ in a length that is polynomial in $n + m$. As in boolean combinational circuits, a *truth assignment* for a boolean formula ϕ is a set of values for the variables of ϕ , and a *satisfying assignment* is a truth assignment that causes it to evaluate to 1. A formula with a satisfying assignment is a *satisfiable* formula. The satisfiability problem asks whether a given boolean formula is satisfiable, which we can express in formal-language terms as

$\text{SAT} = \{\langle \phi \rangle : \phi \text{ is a satisfiable boolean formula}\}.$

As an example, the formula

$$\phi = ((x_1 \rightarrow x_2) \vee \neg((\neg x_1 \leftrightarrow x_3) \vee x_4)) \wedge \neg x_2$$

has the satisfying assignment $\langle x_1 = 0, x_2 = 0, x_3 = 1, x_4 = 1 \rangle$, since

$$\begin{aligned} \phi &= ((0 \rightarrow 0) \vee \neg((\neg 0 \leftrightarrow 1) \vee 1)) \wedge \neg 0 \\ &= (1 \vee \neg(1 \vee 1)) \wedge 1 \\ &= (1 \vee 0) \wedge 1 \\ &= 1, \end{aligned} \tag{34.2}$$

and thus this formula ϕ belongs to SAT.

The naive algorithm to determine whether an arbitrary boolean formula is satisfiable does not run in polynomial time. A formula with n variables has 2^n possible assignments. If the length of $\langle \phi \rangle$ is polynomial in n , then checking every assignment requires $\Omega(2^n)$ time, which is superpolynomial in the length of $\langle \phi \rangle$. As the following theorem shows, a polynomial-time algorithm is unlikely to exist.

Theorem 34.9

Satisfiability of boolean formulas is NP-complete.

Proof We start by arguing that $\text{SAT} \in \text{NP}$. Then we prove that SAT is NP-hard by showing that $\text{CIRCUIT-SAT} \leq_p \text{SAT}$, which by Lemma 34.8 will prove the theorem.

To show that SAT belongs to NP, we show that a certificate consisting of a satisfying assignment for an input formula ϕ can be

verified in polynomial time. The verifying algorithm simply replaces each variable in the formula with its corresponding value and then evaluates the expression, much as we did in equation (34.2) above. This task can be done in polynomial time. If the expression evaluates to 1, then the algorithm has verified that the formula is satisfiable. Thus, SAT belongs to NP.

To prove that SAT is NP-hard, we show that $\text{CIRCUIT-SAT} \leq_p \text{SAT}$. In other words, we need to show how to reduce any instance of circuit satisfiability to an instance of formula satisfiability in polynomial time. We can use induction to express any boolean combinational circuit as a boolean formula. We simply look at the gate that produces the circuit output and inductively express each of the gate's inputs as formulas. We then obtain the formula for the circuit by writing an expression that applies the gate's function to its inputs' formulas.

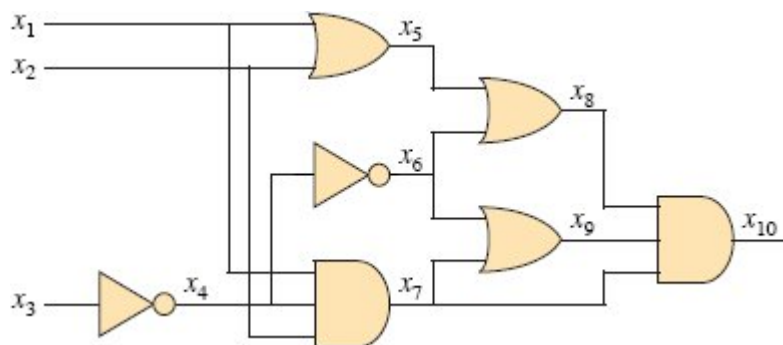


Figure 34.10 Reducing circuit satisfiability to formula satisfiability. The formula produced by the reduction algorithm has a variable for each wire in the circuit and a clause for each logic gate.

Unfortunately, this straightforward method does not amount to a polynomial-time reduction. As Exercise 34.4-1 asks you to show, shared subformulas—which arise from gates whose output wires have fan-out of 2 or more—can cause the size of the generated formula to grow exponentially. Thus, the reduction algorithm must be somewhat more clever.

Figure 34.10 illustrates how to overcome this problem, using as an example the circuit from Figure 34.8(a). For each wire x_i in the circuit C , the formula ϕ has a variable x_i . To express how each gate operates,

construct a small formula involving the variables of its incident wires. The formula has the form of an “if and only if” ($\leftrightarrow$), with the variable for the gate’s output on the left and on the right a logical expression encapsulating the gate’s function on its inputs. For example, the operation of the output AND gate (the rightmost gate in the figure) is $x_{10} \leftrightarrow (x_7 \wedge x_8 \wedge x_9)$. We call each of these small formulas a *clause*.

The formula ϕ produced by the reduction algorithm is the AND of the circuit-output variable with the conjunction of clauses describing the operation of each gate. For the circuit in the figure, the formula is

$$\begin{aligned}\phi = & x_{10} \wedge (x_4 \leftrightarrow \neg x_3) \\ & \wedge (x_5 \leftrightarrow (x_1 \vee x_2)) \\ & \wedge (x_6 \leftrightarrow \neg x_4) \\ & \wedge (x_7 \leftrightarrow (x_1 \wedge x_2 \wedge x_4)) \\ & \wedge (x_8 \leftrightarrow (x_5 \vee x_6)) \\ & \wedge (x_9 \leftrightarrow (x_6 \vee x_7)) \\ & \wedge (x_{10} \leftrightarrow (x_7 \wedge x_8 \wedge x_9)).\end{aligned}$$

Given a circuit C , it is straightforward to produce such a formula ϕ in polynomial time.

Why is the circuit C satisfiable exactly when the formula ϕ is satisfiable? If C has a satisfying assignment, then each wire of the circuit has a well-defined value, and the output of the circuit is 1. Therefore, when wire values are assigned to variables in ϕ , each clause of ϕ evaluates to 1, and thus the conjunction of all evaluates to 1. Conversely, if some assignment causes ϕ to evaluate to 1, the circuit C is satisfiable by an analogous argument. Thus, we have shown that $\text{CIRCUIT-SAT} \leq_p \text{SAT}$, which completes the proof.

■

3-CNF satisfiability

Reducing from formula satisfiability gives us an avenue to prove many problems NP-complete. The reduction algorithm must handle any input

formula, though, and this requirement can lead to a huge number of cases to consider. Instead, it is usually simpler to reduce from a restricted language of boolean formulas. Of course, the restricted language must not be polynomial-time solvable. One convenient language is 3-CNF satisfiability, or 3-CNF-SAT.

In order to define 3-CNF satisfiability, we first need to define a few terms. A *literal* in a boolean formula is an occurrence of a variable (such as x_1) or its negation ($\neg x_1$). A *clause* is the OR of one or more literals, such as $x_1 \vee \neg x_2 \vee \neg x_3$. A boolean formula is in *conjunctive normal form*, or *CNF*, if it is expressed as an AND of clauses, and it's in *3-conjunctive normal form*, or *3-CNF*, if each clause contains exactly three distinct literals.

For example, the boolean formula

$$(x_1 \vee \neg x_1 \vee \neg x_2) \wedge (x_3 \vee x_2 \vee x_4) \wedge (\neg x_1 \vee \neg x_3 \vee \neg x_4)$$

is in 3-CNF. The first of its three clauses is $(x_1 \vee \neg x_1 \vee \neg x_2)$, which contains the three literals x_1 , $\neg x_1$, and $\neg x_2$.

The language 3-CNF-SAT consists of encodings of boolean formulas in 3-CNF that are satisfiable. The following theorem shows that a polynomial-time algorithm that can determine the satisfiability of boolean formulas is unlikely to exist, even when they are expressed in this simple normal form.

Theorem 34.10

Satisfiability of boolean formulas in 3-conjunctive normal form is NP-complete.

Proof The argument from the proof of Theorem 34.9 to show that $\text{SAT} \in \text{NP}$ applies equally well here to show that $3\text{-CNF-SAT} \in \text{NP}$. By Lemma 34.8, therefore, we need only show that $\text{SAT} \leq_p 3\text{-CNF-SAT}$.

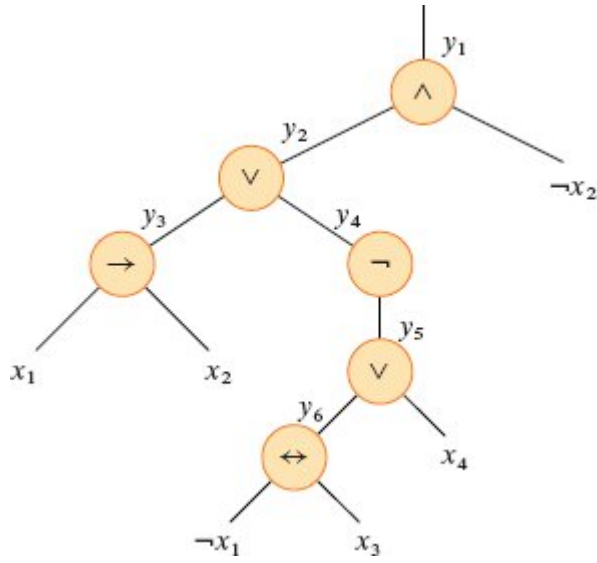


Figure 34.11 The tree corresponding to the formula $\phi = ((x_1 \rightarrow x_2) \vee \neg((\neg x_1 \leftrightarrow x_3) \vee x_4)) \wedge \neg x_2$.

We break the reduction algorithm into three basic steps. Each step progressively transforms the input formula ϕ closer to the desired 3-conjunctive normal form.

The first step is similar to the one used to prove $\text{CIRCUIT-SAT} \leq_p \text{SAT}$ in Theorem 34.9. First, construct a binary “parse” tree for the input formula ϕ , with literals as leaves and connectives as internal nodes. Figure 34.11 shows such a parse tree for the formula

$$\phi = ((x_1 \rightarrow x_2) \vee \neg((\neg x_1 \leftrightarrow x_3) \vee x_4)) \wedge \neg x_2. \quad (34.3)$$

If the input formula contains a clause such as the OR of several literals, use associativity to parenthesize the expression fully so that every internal node in the resulting tree has just one or two children. The binary parse tree is like a circuit for computing the function.

Mimicking the reduction in the proof of Theorem 34.9, introduce a variable y_i for the output of each internal node. Then rewrite the original formula ϕ as the AND of the variable at the root of the parse tree and a conjunction of clauses describing the operation of each node. For the formula (34.3), the resulting expression is

$$\phi' = y_1 \wedge (y_1 \leftrightarrow (y_2 \wedge \neg x_2))$$

$$\begin{aligned}
& \wedge (y_2 \leftrightarrow (y_3 \vee y_4)) \\
& \wedge (y_3 \leftrightarrow (x_1 \rightarrow x_2)) \\
& \wedge (y_4 \leftrightarrow \neg y_5) \\
& \wedge (y_5 \leftrightarrow (y_6 \vee x_4)) \\
& \wedge (y_6 \leftrightarrow (\neg x_1 \leftrightarrow x_3)).
\end{aligned}$$

y_1	y_2	x_2	$(y_1 \leftrightarrow (y_2 \wedge \neg x_2))$
1	1	1	0
1	1	0	1
1	0	1	0
1	0	0	0
0	1	1	1
0	1	0	0
0	0	1	1
0	0	0	1

Figure 34.12 The truth table for the clause $(y_1 \leftrightarrow (y_2 \wedge \neg x_2))$.

The formula ϕ' thus obtained is a conjunction of clauses ϕ'_i , each of which has at most three literals. These clauses are not yet ORs of three literals.

The second step of the reduction converts each clause ϕ'_i into conjunctive normal form. Construct a truth table for ϕ'_i by evaluating all possible assignments to its variables. Each row of the truth table consists of a possible assignment of the variables of the clause, together with the value of the clause under that assignment. Using the truth-table entries that evaluate to 0, build a formula in *disjunctive normal form* (or *DNF*)—an OR of ANDs—that is equivalent to $\neg \phi'_i$. Then negate this formula and convert it into a CNF formula ϕ''_i by using *DeMorgan's laws* for propositional logic,

$$\begin{aligned}
\neg(a \wedge b) &= \neg a \vee \neg b, \\
\neg(a \vee b) &= \neg a \wedge \neg b,
\end{aligned}$$

to complement all literals, change ORs into ANDs, and change ANDs into ORs.

In our example, the clause $\phi'_1 = (y_1 \leftrightarrow (y_2 \wedge \neg x_2))$ converts into CNF as follows. The truth table for ϕ'_1 appears in Figure 34.12. The DNF formula equivalent to $\neg \phi'_1$ is

$$(y_1 \wedge y_2 \wedge x_2) \vee (y_1 \wedge \neg y_2 \wedge x_2) \vee (y_1 \wedge \neg y_2 \wedge \neg x_2) \vee (\neg y_1 \wedge y_2 \wedge \neg x_2).$$

Negating and applying DeMorgan's laws yields the CNF formula

$$\begin{aligned} \phi''_1 = & (\neg y_1 \vee \neg y_2 \vee \neg x_2) \wedge (\neg y_1 \vee y_2 \vee \neg x_2) \\ & \wedge (\neg y_1 \vee y_2 \vee x_2) \wedge (y_1 \vee \neg y_2 \vee x_2), \end{aligned}$$

which is equivalent to the original clause ϕ'_1 .

At this point, each clause ϕ'_i of the formula ϕ' has been converted into a CNF formula ϕ''_i , and thus ϕ' is equivalent to the CNF formula ϕ'' consisting of the conjunction of the ϕ''_i . Moreover, each clause of ϕ'' has at most three literals.

The third and final step of the reduction further transforms the formula so that each clause has *exactly* three distinct literals. From the clauses of the CNF formula ϕ'' , construct the final 3-CNF formula ϕ''' . This formula also uses two auxiliary variables, p and q . For each clause C_i of ϕ'' , include the following clauses in ϕ''' :

- If C_i contains three distinct literals, then simply include C_i as a clause of ϕ''' .
- If C_i contains exactly two distinct literals, that is, if $C_i = (l_1 \vee l_2)$, where l_1 and l_2 are literals, then include $(l_1 \vee l_2 \vee p) \wedge (l_1 \vee l_2 \vee \neg p)$ as clauses of ϕ''' . The literals p and $\neg p$ merely fulfill the syntactic requirement that each clause of ϕ''' contain exactly three distinct literals. Whether $p = 0$ or $p = 1$, one of the clauses is equivalent to $l_1 \vee l_2$, and the other evaluates to 1, which is the identity for AND.
- If C_i contains just one distinct literal l , then include $(l \vee p \vee q) \wedge (l \vee p \vee \neg q) \wedge (l \vee \neg p \vee q) \wedge (l \vee \neg p \vee \neg q)$ as clauses of ϕ''' . Regardless

of the values of p and q , one of the four clauses is equivalent to l , and the other three evaluate to 1.

We can see that the 3-CNF formula ϕ''' is satisfiable if and only if ϕ is satisfiable by inspecting each of the three steps. Like the reduction from CIRCUIT-SAT to SAT, the construction of ϕ' from ϕ in the first step preserves satisfiability. The second step produces a CNF formula ϕ'' that is algebraically equivalent to ϕ' . Then the third step produces a 3-CNF formula ϕ''' that is effectively equivalent to ϕ'' , since any assignment to the variables p and q produces a formula that is algebraically equivalent to ϕ'' .

We must also show that the reduction can be computed in polynomial time. Constructing ϕ' from ϕ introduces at most one variable and one clause per connective in ϕ . Constructing ϕ'' from ϕ' can introduce at most eight clauses into ϕ'' for each clause from ϕ' , since each clause of ϕ' contains at most three variables, and the truth table for each clause has at most $2^3 = 8$ rows. The construction of ϕ''' from ϕ'' introduces at most four clauses into ϕ''' for each clause of ϕ'' . Thus the size of the resulting formula ϕ''' is polynomial in the length of the original formula. Each of the constructions can be accomplished in polynomial time.

■

Exercises

34.4-1

Consider the straightforward (nonpolynomial-time) reduction in the proof of Theorem 34.9. Describe a circuit of size n that, when converted to a formula by this method, yields a formula whose size is exponential in n .

34.4-2

Show the 3-CNF formula that results upon using the method of Theorem 34.10 on the formula (34.3).

34.4-3

Professor Jagger proposes to show that $\text{SAT} \leq_p \text{3-CNF-SAT}$ by using only the truth-table technique in the proof of Theorem 34.10, and not the other steps. That is, the professor proposes to take the boolean formula ϕ , form a truth table for its variables, derive from the truth table a formula in 3-DNF that is equivalent to $\neg\phi$, and then negate and apply DeMorgan's laws to produce a 3-CNF formula equivalent to ϕ . Show that this strategy does not yield a polynomial-time reduction.

34.4-4

Show that the problem of determining whether a boolean formula is a tautology is complete for co-NP. (*Hint*: See Exercise 34.3-7.)

34.4-5

Show that the problem of determining the satisfiability of boolean formulas in disjunctive normal form is polynomial-time solvable.

34.4-6

Someone gives you a polynomial-time algorithm to decide formula satisfiability. Describe how to use this algorithm to find satisfying assignments in polynomial time.

34.4-7

Let 2-CNF-SAT be the set of satisfiable boolean formulas in CNF with exactly two literals per clause. Show that $2\text{-CNF-SAT} \in P$. Make your algorithm as efficient as possible. (*Hint*: Observe that $x \vee y$ is equivalent to $\neg x \rightarrow y$. Reduce 2-CNF-SAT to an efficiently solvable problem on a directed graph.)

34.5 NP-complete problems

NP-complete problems arise in diverse domains: boolean logic, graphs, arithmetic, network design, sets and partitions, storage and retrieval, sequencing and scheduling, mathematical programming, algebra and number theory, games and puzzles, automata and language theory, program optimization, biology, chemistry, physics, and more. This section uses the reduction methodology to provide NP-completeness